PRACTICAL : 1

DESIGN A SIMPLE RATIONAL AGENT BY DEFINING, PEAS FOR VACUUM CLEANER ENVIRONMENT AND AUTONOMOUS TAXI.

CODE:

```
 rooms={}
rooms['A']=input("Enter status of Room A(Clean/Dirty):")
rooms['B']=input("Enter status of Room B(Clean/Dirty):")
if rooms['A']=="Cleam" and rooms['B']=="Clean":
    print("\nAll rooms are clean. Task Completed.")


else:
    if rooms['A']=="Dirty":
        print("\nRoom A is Dirty")
        print("Action:Suck")
        rooms['A']="Clean"
        print("Rooms A Cleaned")


    else:
        print("\nRoom A is already Clean")
    if rooms['B']=="Dirty":
        print("\nRoom B is Dirty")
        print("Action:Suck")
        rooms['B']="Clean"
```

```
    print("Room B Cleaned")
else:
    print("Room B is Already Clean")
print("\nAll Rooms are clean . Task Completed")
```

OUTPUT:

```
Enter status of Room A(Clean/Dirty):Dirty
Enter status of Room B(Clean/Dirty):Dirty

Room A is Dirty
Action:Suck
Rooms A Cleaned

Room B is Dirty
Action:Suck
Room B Cleaned

All Rooms are clean . Task Completed
```

PRACTICAL : 2

GIVEN AN INITIAL CONFIGURATION OF THE 8-PUZZLE AND A GOAL CONFIGURATION, WRITE A PYTHON PROGRAM TO FIND THE SHORTEST SEQUENCE OF MOVES TO REACH THE GOAL STATE USING BREADTH-FIRST SEARCH (BFS).

CODE:

```
from collections import deque

def print_state(state):
    for i in range (0,9,3):
        print(state[i],state[i+1],state[i+2])
    print()


def get_neighbors(state):
        neighbors=[]


        blank_index=state.index(0)


        moves={
            "Up":-3,
            "Down":3,
            "Left":-1,
            "Right":1
        }


        for move,position_change in moves.items():
            new_index=blank_index+position_change
```

```
        if move=="Up" and blank_index<3:
            continue
        if move=="Down" and blank_index>5:
            continue
        if move=="Left" and blank_index%3==0:
            continue
        if move=="Right" and blank_index%3==2:  continue

        new_state=list(state)
        new_state[blank_index],new_state[new_index]=(
            new_state[new_index],
            new_state[blank_index]
        )
        neighbors.append((tuple(new_state),move))
    return neighbors
def bfs(initial_state,goal_state):
    queue=deque()
    queue.append((initial_state,[]))

    visited=set()
    visited.add(initial_state)

    while queue:
        current_state,path=queue.popleft()

        if current_state==goal_state:
```

```
            return path

        for neighbor, move in get_neighbors(current_state):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor,path+[(move,neighbor)]))
    return None

initial_state=(1,2,3,
            4,0,6,
            7,5,8)

goal_state=(1,2,3,
        4,5,6,
        7,8,0)
print("Initial State:")
print_state(initial_state)

print("Goal State:")
print_state(goal_state)

solution=bfs(initial_state,goal_state)
if solution:
    print("Shortest sequence of moves:")
    current_step=1
    for move,state in solution:
```

```
        print("Step",current_step,"-Move:",move)
        print_state(state)
        current_step+=1
    print("Goal reached successfully.")
    print("Total number of moves:",len(solution))
else:
    print("No solution found")
```

OUTPUT:

```
Initial State:
1 2 3
4 0 6
7 5 8

Goal State:
1 2 3
4 5 6
7 8 0

Shortest sequence of moves:
Step 1 -Move: Down
1 2 3
4 5 6
7 0 8

Step 2 -Move: Right
1 2 3
4 5 6
7 8 0

Goal reached successfully.
Total number of moves: 2
```

PRACTICAL : 3

GIVEN TWO WATER JUGS OF 4 LITRES AND 3 LITRES CAPACITI WRITE A PYTHON PROGRAM TO OBTAIN EXACTLY 2 LITRES IN ON JUG USING DEPTH-FIRST SEARCH (DFS). VARY CAPACITY OF LUGS.

CODE:

```
j1 = int(input("Enter capacity of jug 1: "))
j2 = int(input("Enter capacity of jug 2: "))
goal = int(input("Enter target amount: "))


visited = set()
def dfs(x, y, path):
    if x == goal or y == goal:
        path.append((x, y))
        print("\nGoal Achieved Successfully!")
        print("Solution Path:")
        for state in path:
            print(state)
        return True

    if (x, y) in visited:
        return False

    visited.add((x, y))
    path.append((x, y))
    next_states = [
```

```
        (j1, y),
        (x, j2),
        (0, y),
        (x, 0),
        (x - min(x, j2 - y), y + min(x, j2 - y)),
        (x + min(y, j1 - x), y - min(y, j1 - x))
    ]
    for state in next_states:
        if dfs(state[0], state[1], path.copy()):
            return True
    return False
if not dfs(0, 0, []):
    print("No solution found.")
```

OUTPUT:

```
Enter capacity of jug 1: 4
Enter capacity of jug 2: 3
Enter target amount: 2

Goal Achieved Successfully!
Solution Path:
(0, 0)
(4, 0)
(4, 3)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
```

PRACTICAL : 4

GIVEN A WEIGHTED GRAPH REPRESENTING CITIES AND DISTANCES BETWEEN THEM, WRITE A PYTHON PROGRAM TO FIND THE LEAST-COST PATH FROM ARAD TO BUCHAREST USING UNIFORM COST SEARCH (UCS).

[PROVIDE ANY OTHER WEIGHTED GRAPH FOR APPLYING UCS]

```
import heapq
graph = {
    'Mumbai': [('Pune', 150), ('Hyderabad', 710)],
    'Pune': [('Mumbai', 150), ('Bangalore', 840)],
    'Hyderabad': [('Mumbai', 710), ('Bangalore', 570)],
    'Bangalore': []
}
def ucs(start, goal):
    queue = [(0, start, [start])]
    visited = []

    while queue:
        cost, city, path = heapq.heappop(queue)

        if city == goal:
            return cost, path

        if city not in visited:
            visited.append(city)
```

```
        for neighbor, distance in graph[city]:
            heapq.heappush(
                queue,
                (cost + distance,
                 neighbor,
                 path + [neighbor])
            )


cost, path = ucs("Mumbai","Bangalore")



print("Least-Cost path:")
print("Path:", " -> ".join(path))
print("Total Distance:",cost,"km")
```

OUTPUT:

```
Least-Cost path:
Path: Mumbai -> Pune -> Bangalore
Total Distance: 990 km
```

PRACTICAL: 5

GIVEN AN INITIAL CONFIGURATION OF THE 8-PUZZLE AND A GOAL SOLUTION USING GREEDY BEST-FIRST SEARCH WITH THE CONFIGURATION, WRITE A PYTHON PROGRAM TO FIND MANHATTAN DISTANCE HEURISTIC.

```
import heapq

def print_state(state):
    for i in range(0, 9, 3):
        print(state[i], state[i + 1], state[i + 2])
    print()



def heuristic(state, goal):
    distance = 0
    for num in range(1, 9):
        current = state.index(num)
        target = goal.index(num)
        x1, y1 = current // 3, current % 3
        x2, y2 = target // 3, target % 3
        distance += abs(x1 - x2) + abs(y1 - y2)
    return distance



def get_neighbour(state):
    neighbour = []
```

```
    blank = state.index(0)
    moves = [-3, 3, -1, 1]

    for move in moves:
        new = blank + move

        if new >= 0 and new < 9:
            if blank % 3 == 0 and move == -1:
                continue
            if blank % 3 == 2 and move == 1:
                continue

            temp = list(state)
            temp[blank], temp[new] = temp[new], temp[blank]
            neighbour.append(tuple(temp))

    return neighbour



def greedy(initial, goal):
    queue = []
    heapq.heappush(
        queue,
        (heuristic(initial, goal), initial, [])
    )
```

```
    visited = set()

    while queue:
        h, state, path = heapq.heappop(queue)

        if state == goal:
            return path + [state]

        if state not in visited:
            visited.add(state)

            for next_state in get_neighbour(state):
                heapq.heappush(
                    queue,
                    (
                        heuristic(next_state, goal),
                        next_state,
                        path + [state]
                    )
                )

    return None
```

```
initial = (
    1, 2, 3,
    4, 0, 6,
    7, 5, 8
)

goal = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)

print("Initial State")
print_state(initial)

print("Goal State")
print_state(goal)

solution = greedy(initial, goal)

print("Solution:")

if solution:
    for step, state in enumerate(solution):
        print("Step", step)
```

```
        print_state(state)


    print("Goal reached successfully.")
else:
    print("No solution found.")
```

OUTPUT:

```
Initial State
1 2 3
4 0 6
7 5 8

Goal State
1 2 3
4 5 6
7 8 0

Solution:
Step 0
1 2 3
4 0 6
7 5 8

Step 1
1 2 3
4 5 6
7 0 8

Step 2
1 2 3
4 5 6
7 8 0

Goal reached successfully.
```

Practical-6

GIVEN AN INITIAL CONFIGURATION OF THE 8-PUZZLE AND A GOAL CONFIGURATION, WRITE A PYTHON PROGRAM TO FIND THE SHORTEST PATH USING A SEARCH WITH THE MANHATTAN DISTANCE HEURISTIC.

```
import heapq
GOAL = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 0)
)

def manhattan(state):
    distance = 0
    for i in range(3):
        for j in range(3):
            value = state[i][j]
            if value != 0:
                goal_x = (value - 1) // 3
                goal_y = (value - 1) % 3
                distance += abs(i - goal_x) + abs(j - goal_y)
    return distance

def find_blank(state):
    for i in range(3):
        for j in range(3):
```

```
            if state[i][j] == 0:
                return i, j

def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dx, dy in directions:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors

def astar(start):
    pq = []
    heapq.heappush(pq, (manhattan(start), 0, start, []))

    visited = set()

    while pq:
```

```
        f, g, state, path = heapq.heappop(pq)

        if state in visited:
            continue

        visited.add(state)
        path = path + [state]

        if state == GOAL:
            return path

        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                h = manhattan(neighbor)
                heapq.heappush(pq, (g + 1 + h, g + 1, neighbor, path))

    return None

def print_state(state):
    for row in state:
        print(row)
    print()
initial = (
    (1, 2, 3),
    (4, 0, 6),
```

```
    (7, 5, 8)
)
solution = astar(initial)
if solution:
    print("Solution found in", len(solution) - 1, "moves:\n")
    for step, state in enumerate(solution):
        print("Step", step)
        print_state(state)
else:
    print("No solution exists.")
```

OUTPUT:

```
Solution found in 2 moves:

Step 0
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Step 1
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step 2
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

PRACTICAL : 7

GIVEN A WEIGHTED GRAPH REPRESENTING CITIES AND DISTANO BETWEEN THEM, WRITE A PYTHON PROGRAM TO FIND TH SHORTEST PATH FROM ARAD TO BUCHAREST USING A* SEARCH WITH A HEURISTIC FUNCTION (STRAIGHT-LINE DISTANCE DESTINATION).

```
import heapq


graph = {
    'Arad': [('Zerind',75), ('Sibiu',140), ('Timisoara',118)],
    'Zerind': [('Arad',75), ('Oradea',71)],
    'Oradea': [('Zerind',71), ('Sibiu',151)],
    'Sibiu': [('Arad',140), ('Oradea',151), ('Fagaras',99), ('Rimnicu Vilcea',80)],
    'Timisoara': [('Arad',118), ('Lugoj',111)],
    'Lugoj': [('Timisoara',111), ('Mehadia',70)],
    'Mehadia': [('Lugoj',70), ('Drobeta',75)],
    'Drobeta': [('Mehadia',75), ('Craiova',120)],
    'Craiova': [('Drobeta',120), ('Rimnicu Vilcea',146), ('Pitesti',138)],
    'Rimnicu Vilcea': [('Sibiu',80), ('Craiova',146), ('Pitesti',97)],
    'Fagaras': [('Sibiu',99), ('Bucharest',211)],
    'Pitesti': [('Rimnicu Vilcea',97), ('Craiova',138), ('Bucharest',101)],
    'Bucharest': []
}


heuristic = {
    'Arad': 366,
    'Zerind': 374,
```

```
    'Oradea': 380,
    'Sibiu': 253,
    'Timisoara': 329,
    'Lugoj': 244,
    'Mehadia': 241,
    'Drobeta': 242,
    'Craiova': 160,
    'Rimnicu Vilcea': 193,
    'Fagaras': 176,
    'Pitesti': 100,
    'Bucharest': 0
}

def astar(start, goal):
    queue = []
    heapq.heappush(queue, (heuristic[start], 0, start, [start]))
    visited = set()

    while queue:
        f_cost, g_cost, city, path = heapq.heappop(queue)

        if city == goal:
            return g_cost, path

        if city not in visited:
            visited.add(city)
```

```
        for neighbor, distance in graph[city]:
            new_g = g_cost + distance
            new_f = new_g + heuristic[neighbor]
            heapq.heappush(queue, (new_f, new_g, neighbor, path + [neighbor]))

    return None, []

cost, path = astar('Arad', 'Bucharest')

print("Shortest path using A* Search:")
print("Path:", " => ".join(path))
print("Total Cost:", cost)
```

OUTPUT:

```
Shortest path using A* Search:
Path: Arad => Sibiu => Rimnicu Vilcea => Pitesti => Bucharest
Total Cost: 418
```

PRACTICAL : 8

WRITE A PYTHON PROGRAM TO COMPUTE THE PROBABILITY THAT THE PERSON ACTUALLY HAS THE DISEASE GIVEN THAT THE TEST RESULT IS POSITIVE.

```
P_D = 0.01

P_Pos_D = 0.99

P_Pos_NotD = 0.05

P_NotD = 1-P_D

P_D_Pos = (P_Pos_D*P_D)/((P_Pos_D*P_D)+(P_Pos_NotD*P_NotD))

print("Probability that person actually has disease:")

print(round(P_D_Pos * 100,2),"%")
```

OUTPUT:

```
Probability that person actually has disease:
16.67 %
```

PRACTICAL : 9

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB

data = {
    'Outlook': ['Sunny', 'Overcast', 'Rainy', 'Rainy', 'Rainy', 'Overcast'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal'],
    'Windy': ['False', 'True', 'False', 'False', 'False', 'True'],
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No']
}

df = pd.DataFrame(data)

le = LabelEncoder()
for col in df.columns:
    df[col] = le.fit_transform(df[col])

X = df[['Outlook', 'Temperature', 'Humidity', 'Windy']]
y = df['Play']

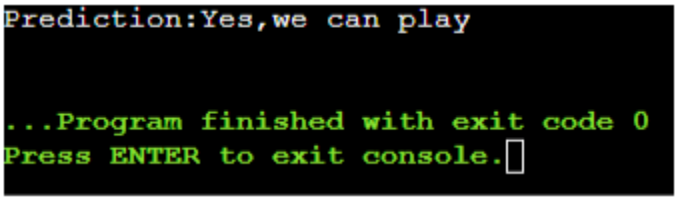
model = GaussianNB()
model.fit(X, y)
```

```
test = pd.DataFrame(
    [[2, 0, 0, 1]],
    columns=['Outlook', 'Temperature', 'Humidity', 'Windy']
)

prediction = model.predict(test)

if prediction[0] == 1:
    print("Prediction: Yes, we can play")
else:
    print("Prediction: No, we cannot play")
```

Output:



Practical-:10

```
from pgmpy.models import DiscreateBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])

cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999],[0.001]],
    state_names={"Burglary":["False","True"]}
    )

cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
```

```
    values=[[0.998],[0.002]],
    state_names={"Earthquake":["False","True"]}
    )

cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values={
        [0.999,0.06,0.05],
        [0.0001,0.29,0.94,0.95]
    },
        evidence=["Burglary","Earthquake"],
    evidence_card=[2,2],
    state_names={
        "Alarm":["False","Ture"],
        "Burglary":["False","Ture"],
        "Earthquake":["False","True"]
    }
)

cpd_john = TabularCPD(
    variabl="jhonCalls",
    variable_card=2,
```

```
    values=[
        [0.95,0.10],
        [0.05,0.09]
        ],
        evidence=["Alarm"],
    avidence_card=[2,2],
    state_names={
        "jhonCalls":["False","True"],
        "Alarm":["False","True"],
    }
)

cpd_mary = TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99,0.30],
        [0.01,0.70]
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "MaryCalls":["False","True"],
```

```
        "Alarm":["False","True"]
    }
)

model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm,
    cpd_john,
    cpd_mary
)

print("is Bayesian Network valid?",model.check_model())
inference = VariableElimination(model)
result = inference.query(
    variables=["Burglary"],
    evidence={
        "johnCalls": "True",
        "MarryCalls": "True"
    }
)

print("\nProbability of Burglary given jhonCalss=True and MaryCalls=True:")
```

pirnt(result)

Output:-

```
Is Bayesian Network valid? True

Probability of Burglary given JohnCalls=True and MaryCalls=True:
+-----------------+-----------------+
| Burglary        |   phi(Burglary) |
+=================+=================+
| Burglary(False) |          0.7158 |
+-----------------+-----------------+
| Burglary(True)  |          0.2842 |
+-----------------+-----------------+
>>>
```